

Vector Embeddings Are Lossy. Here's What to Do About It.

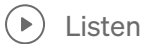
AI systems aren't perfect (GASP!) and these are some of the reasons why.



Brian Godsey · Follow

Published in Towards Data Science

17 min read · 3 days ago

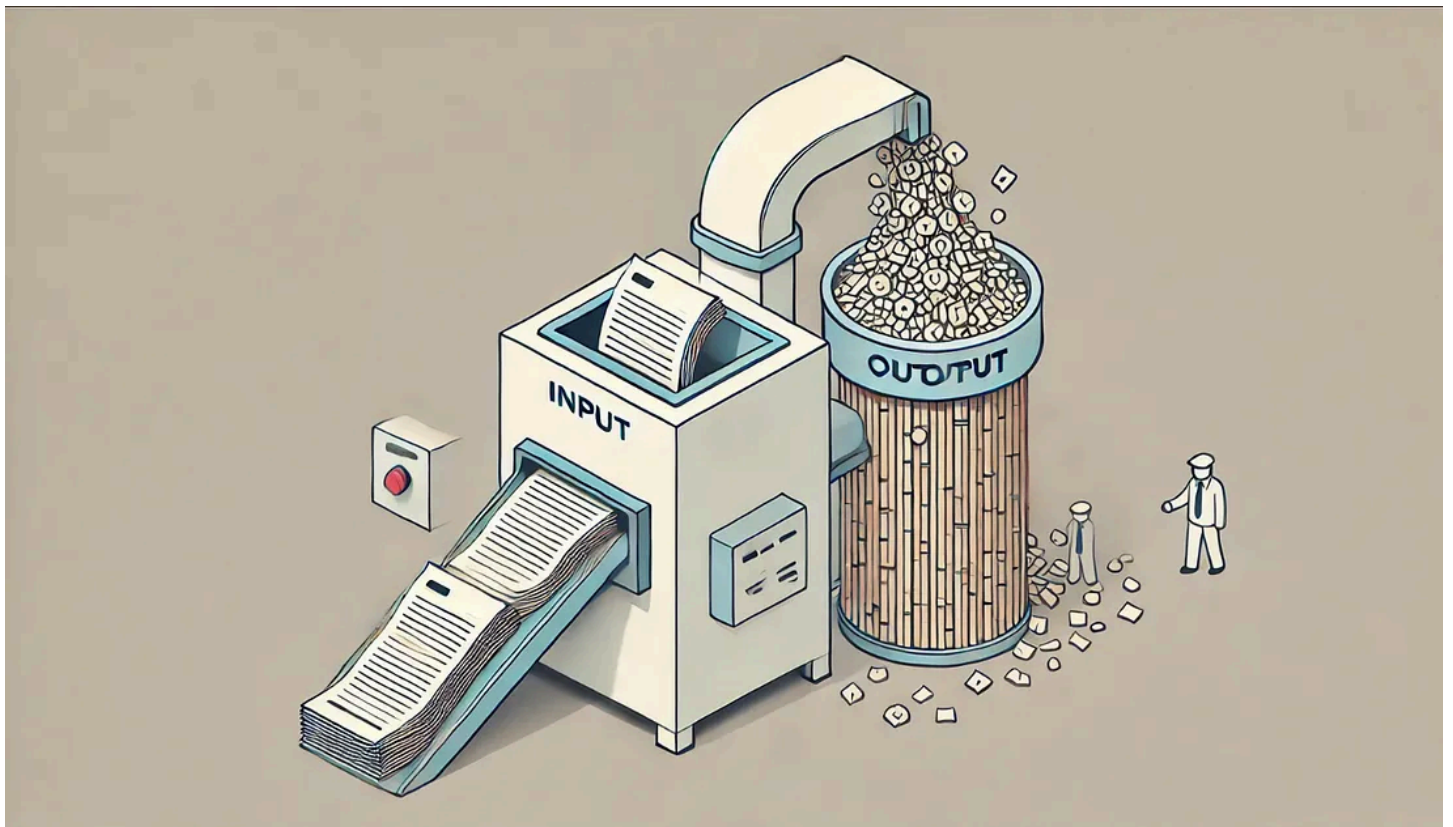


Listen



Share

... More



Created by the author using DALL-E. Some information loss was involved.

As we bring enterprise AI systems into production, we shouldn't expect them to function in the same way as search engines, or as databases of exact words and phrases. Yes, AI systems often feel like they have the same search capabilities as a (non-vector) document store or search engine, but under the hood, they work in a

very different way. If we try to use an AI system — consisting mainly of a vector store and LLM — as if the data were structured and the query results were exact, we could get some unexpected and disappointing results.

AI systems do not generally “memorize” the data itself. Even RAG systems, which preserve the full texts of the main document set, use vector search for retrieval, a process that is powerful but imperfect and inexact. Some amount of information is “lost” in virtually all AI systems.

This, of course, leads to the question: what should we do about this information loss? The short answer: we should recognize the use cases that benefit from the preservation of certain types of information, and deliberately preserve that information, where possible. Often, this means incorporating deterministic, structured, non-AI software processes into our systems, with the goal of preserving structure and exactness where we need it.

In this article, we discuss the nuances of the problem and some potential solutions. There are many possibilities for addressing specific problems, such as implementing a knowledge graph to structure topics and concepts, integrating keyword search as a feature alongside the vector store, or tailoring the data processing, chunking, and loading to fit your exact use case. In addition to those, as we discuss below, one of the most versatile and accessible methods to layer structure onto a vector store of unstructured documents is to use document metadata to navigate the knowledge base in structured ways. A vector graph of document links and tags can be a powerful, lightweight, efficient, and easy-to-implement way of layering useful structure back into your unstructured data.

AI systems are mostly unstructured, inexact, and fuzzy

It is a given that some information loss will occur in systems built around large amounts of unstructured data. Diagnosing where, how, and why this information loss occurs for your use case can be a helpful exercise leading to improved systems and better applications.

With respect to information preservation and loss in AI systems, the three most important things to note are:

1. Vector embeddings do not preserve 100% of the information in the original text.

2. LLMs are non-deterministic, meaning text generation includes some randomness.
3. It is hard to predict what will be lost and what will be preserved.

The first of these means that some information is lost from our documents on their way into the vector store; the second means that some information is randomized and inexact after retrieval on the way through the LLM; and the third means that we probably don't know when we might have a problem or how big it will be.

Below, we dive deeper into point one above: that vector embeddings themselves are lossy. We examine how lossy embeddings are generally unavoidable, how it affects our applications, and how — rather than trying to recover or prevent the loss within the LLM framework — it is much more valuable for us to maintain awareness of the process of information loss and add structured layers of information into our AI systems that suit our specific use cases and build upon the power of our existing vector-embedding-powered AI systems.

Next, let's dig a little deeper into the question of how information loss works in vector embeddings.

Vector embeddings are lossy

Vector representations of text — the embeddings that LLMs work with — contain vast amounts of information, but this information is necessarily approximate. Of course, it is possible to build a deterministic LLM whose vectors represent precise texts that can be generated, word-for-word, over and over given the same initial vector. But, this would be limited and not very helpful. For an LLM and its vector embeddings to be useful in the ways we work with them today, the embedding process needs to capture nuanced concepts of language more than the exact words themselves. We want our LLMs to “understand” that two sentences that say essentially the same thing represent the same set of concepts, regardless of the specific words used. “I like artificial intelligence” and “AI is great” tell us basically the same information, and the main role of vectors and embeddings is to capture this information, not memorize the words themselves.

Vector embeddings are high-dimensional and precise, allowing them to encapsulate complex ideas within a vast conceptual space. These dimensions can number in the hundreds or even thousands, each subtly encoding aspects of language — from syntax and semantics to pragmatics and sentiment. This high dimensionality

enables the model to navigate and represent a broad spectrum of ideas, making it possible to grasp intricate and abstract concepts embedded within the text.

Despite the precision of these embeddings, text generation from a given vector remains a non-deterministic process. This is primarily due to the probabilistic nature of the models used to generate text. When an LLM generates text, it calculates the probability of each possible word that could come next in a sequence, based on the information contained in the vector. This process incorporates a level of randomness and contextual inference, which means that even with the same starting vector, the output can vary each time text is generated. This variability is crucial for producing natural-sounding language that is adaptable to various contexts but also means that exact reproduction of text is not always possible.

While vectors capture the essence of the text's meaning, specific words and information are often lost in the vector embedding process. This loss occurs because the embeddings are designed to generalize from the text, capturing its overall meaning rather than the precise wording. As a result, minor details or less dominant themes in the text may not be robustly represented in the vector space. This characteristic can lead to challenges when trying to retrieve specific facts or exact terms from a large corpus, as the system may prioritize overall semantic similarity over exact word matches.

Two of the most common ways that we may have problems with information loss are:

1. Tangential details contained in a text are “lost” among the semantic meaning of the text as a whole.
2. The significance of specific keywords or phrases are “lost” during the embedding process into semantic space.

The first of these two cases concerns the “loss” of actual details contained within a document (or chunk) because the embedding doesn't capture it very well. The second case mostly concerns the loss of specific wording of the information and not necessarily any actual details. Of course, both types of loss can be significant and problematic in their own ways.

The very recent article [Embeddings are Kind of Shallow](#) (also in this publication) gives a lot of fun examples of ways that embeddings lose or miss details, by way of

testing search and retrieval results among relatively small text chunks across a few popular embeddings algorithms.

Next let's look at some live examples of how each of these two types of loss works, with code and data.

Case study: AI product pages

For this case study, I created a dataset of product pages for the website of a fictional company called **Phrase AI**. Phrase AI builds LLMs and provides them as a service. Its first three products are **Phrase Flow**, **Phrase Forge**, and **Phrase Factory**. Phrase Flow is the company's flagship LLM, suitable for general use cases, but exceptionally good at engaging, creative content. The other two products are specialized LLMs with their own strengths and weaknesses.

The dataset of HTML documents consists of a main home page for `phrase.ai` (fictional), one product page per LLM (three total), and four more pages on the site: **Company Purpose**, **Ongoing Work**, **Getting Started**, and **Use Cases**. The non-product pages center mostly on the flagship product, Phrase Flow, and each of the product pages focuses on the corresponding LLM. Most of the text is standard web copy, generated by ChatGPT, but there are a few features of the documents that are important for our purposes here.

Most importantly, **each product page contains important information about the flagship product, Phrase Flow**. Specifically, each of the product pages for the two specialized LLMs contains a warning not to use the Phrase Flow model for specific purposes. The bottom of the Phrase Forge product page contains the text:

```
Special Strengths: Phrase Forge is exceptionally good at creating a
complete Table of Contents, a task that general models like Phrase
Flow do not excel at. Do not use Phrase Flow for Tables of Contents.
```

And, the bottom of the Phrase Factory product page contains the text:

```
Special Strengths: Phrase Factory is great for fact-checking and
preventing hallucinations, far better than more creative models like
```

Phrase Flow. Do not use Phrase Flow for documents that need to be factual.

Of course, it is easy to argue that Phrase AI should have these warnings on their Phrase Flow page, and not just on the pages for the other two products. But, I think we all have seen examples of critical information being in the “wrong” place on a website or in documentation, and we still want our RAG systems to work well even when some information is not in the best possible place in the document set.

While this dataset is fabricated and very small, we have designed it to be clearly illustrative of issues that can be quite common in real-life cases, which can be hard to diagnose on larger datasets. Next, let's examine these issues more closely.

Tangential details can get buried in semantic space

Vector embeddings are lossy, as I've discussed above, and it can be hard to predict which information will be lost in this way. All information is at risk, but some more than others. Details that relate directly to the main topic of a document are generally more likely to be captured in the embedding, whereas details that stray from the main topic are more likely to be lost or hard to find using vector search.

In the case study above, we highlighted two pieces of information about the Phrase Flow product that are found on the product pages for the other two models. These two warnings are quite strong, using the wording, “Do not use Phrase Flow for...”, and could be critical to answering queries about the capabilities of the Phrase Flow model. But, they appear in documents that are not primarily about Phrase Flow, and are therefore “tangential” details with respect to those documents.

To test how a typical RAG system might handle these documents, we built a RAG pipeline using LangChain's `GraphVectorStore`, and OpenAI APIs. Code can be found in [this Colab notebook](#).

We can query the system about the weaknesses of Phrase Flow and get the following results:

Question:

What are some weaknesses of Phrase Flow?

Retrieved documents:

['https://phrase.ai/products/phraseflow',

```
'https://phrase.ai/usecases',  
'https://phrase.ai/ongoingwork',  
'https://phrase.ai/purpose']
```

LLM response:

The provided context does not mention any specific weaknesses of Phrase Flow.

Note that we set up the retriever to fetch the top four documents — half of the eight total documents. The two product pages with the warnings about Phrase Flow do not appear in these results, and so the LLM response has no context about weaknesses of Phrase Flow. We get similar results for other wordings and other queries asking about “issues” or “problems” with Phrase Flow. See the notebook for more examples.

If we don't limit our retriever to four documents, we get the following results, including the retrieval rankings of all eight documents:

Question:

What are some weaknesses of Phrase Flow?

Retrieved documents:

```
['https://phrase.ai/products/phraseflow',  
'https://phrase.ai/usecases',  
'https://phrase.ai/ongoingwork',  
'https://phrase.ai/purpose',  
'https://phrase.ai/gettingstarted',  
'https://phrase.ai/products/phraseforge',  
'https://phrase.ai/products/phrasefactory',  
'https://phrase.ai']
```

LLM response:

Phrase Flow is not ideal for creating Tables of Contents or for documents that need to be factual. It may not excel in generating detailed, structured writing for technical guides, reports, or research papers, as these tasks are better suited for Phrase Forge. Additionally, Phrase Flow might not be the best choice for answering questions quickly or retrieving information in real-time, as Phrase Factory is specifically designed for those needs.

Notably, the product pages for Phrase Forge and Phrase Factory — the two pages with the warnings about Phrase Flow, are ranked 6th and 7th in the retrieved

results. Roughly speaking, those two warnings are less discoverable because they have been “buried” in vector embeddings that mainly capture information relevant to the topics of each page: either Phrase Forge or Phrase Factory.

However, if we rephrase the query to semantics that more closely align with the phrasing of the warnings, “Do not use Phrase Flow...”, we get the results we want and expect:

Question:

When should I not use Phrase Flow?

Retrieved documents:

```
['https://phrase.ai/products/phraseflow',  
'https://phrase.ai/products/phraseforge',  
'https://phrase.ai/products/phrasefactory',  
'https://phrase.ai/usecases']
```

LLM response:

You should not use Phrase Flow for creating documents that need to be factual, such as technical guides, research papers, white papers, or any text that requires precise language and detailed structure. For these types of documents, Phrase Forge would be more appropriate. Additionally, Phrase Flow is not ideal for creating Tables of Contents.

Here, retrieval seems to be sensitive to the particular wording of the query, and the phrase “not use Phrase Flow” nudges us closer to the documents that we need, in semantic vector space. But, we wouldn’t know this beforehand. We wouldn’t know exactly what we are looking for, and we are relying on our RAG stack to help us find it.

Further below, we discuss some possible solutions for addressing this type of buried information mainly due to lossy semantic vectors. But first, let’s look at another way that lossy vectors can cause counter-intuitive behavior in RAG systems.

Vector retrieval is not a search engine or keyword search

Many users tend to expect AI and RAG systems to be able to match names, keywords, and phrases exactly. We are used to traditional search engines, and we have the distinct feeling that AI is so much more powerful, so why wouldn’t it be able to find the exact matches that we want?

As previously discussed, vector search operates fundamentally differently from search engines, text search, and other pre-AI methods for querying data — all of which operate on search algorithms for exact matches, with limited fuzzy search operators. While vector search does often locate specific words and phrases, there is no guarantee, because vectors are in semantic space and embedding text into vectors is a lossy process.

The words and phrases that are most likely to experience some kind of information loss are those whose semantic meanings are unclear or ambiguous. We included examples of this in the dataset for our case study. Specifically, the following text appears at the end of the Ongoing Work page for our fictional company, Phrase AI:

COMING SOON: Our newest specialized models Flow Factory, Flow Forge, and Factory Forge are in beta and will be released soon!

This is the only mention in the dataset of these forthcoming models. Not only are “Flow Factory”, “Flow Forge”, and “Factory Forge” confusing remixes of other names in the product line, but they are also simple combinations of dictionary words. “Flow Factory”, for example, has a semantic meaning beyond the product name, including some combination of the well-known meanings of the words “flow” and “factory” separately. Contrast this with a proprietary spelling such as “FloFaktoree”, which has virtually no real inherent semantic meaning and would likely be treated by an AI system in a very different way — and would likely be more discoverable as a term that does not blend in with anything else.

If we ask specifically about “Flow Forge” or “Factory Forge”, we get results like this:

Question:

What is Flow Forge?

Retrieved documents:

```
['https://phrase.ai/products/phraseforge',  
'https://phrase.ai/usecases',  
'https://phrase.ai/ongoingwork',  
'https://phrase.ai/products/phraseflow']
```

LLM response:

Flow Forge is a new specialized model that is currently in beta and will be released soon.

So the system successfully retrieves the one document — the page about Ongoing Work — that contains a reference to “Flow Forge”, but it is the 3rd-ranked retrieved document. In semantic space, two documents appear more relevant, even though they don't mention “Flow Forge” at all. In large datasets, it is easy to imagine names, terms, keywords, and phrases getting buried or “lost” in semantic space in hard-to-diagnose ways.

What to do about lossy vectors?

We have been discussing lossy vectors as if they are a problem that needs to be solved. Sure, there are problems that stem from vectors being “lossy”, but vector search and AI systems depend on using vector embeddings to translate documents from text into semantic space, a process that necessarily loses some textual information, but gains all of the power of semantic search. So “lossy” vectors are a feature, not a bug. Even if lossy vectors are not a bug, it helps for us to understand their advantages, disadvantages, and limits in order to know what they can do as well as when they might surprise us with unexpected behavior.

If any of the issues described above ring true for your AI systems, the root cause is probably not that vector search is performing poorly. You could try to find alternate embeddings that work better for you, but this is a complex and opaque process, and there are usually much simpler solutions.

The root cause of the above issues is that we are often trying to make vector search do things that it was not designed to do. The solution, then, is to build functionality into your stack, adding the capabilities that you need for your specific use case, alongside vector search.

Alternate chunking and embedding methods

There are many options when it comes to chunking documents for loading as well

as embedding and searching them. We have a great guide in Semantic Search 101 that

[Open in app](#)

Medium

 Search

 25



Optimized chunking strategy — The chunking strategy dictates how text is segmented into chunks for processing and retrieval. Optimizing chunking goes

beyond mere size or boundary considerations; it involves segmenting texts in a way that aligns with thematic elements or logical divisions within the content. This approach ensures that each chunk represents a complete thought or topic, which facilitates more coherent embeddings and improves the retrieval accuracy of the RAG system.

Multi-vector embedding techniques — Standard embedding practices often reduce a passage to a single vector representation, which might not capture the passage's multifaceted nature. Multi-vector embedding techniques address this limitation by employing models to generate several embeddings from one passage, each corresponding to different interpretations or questions that the passage might answer. This strategy enhances the dimensional richness of the data representation, allowing for more precise retrieval across varied query types.

ColBERT: Token-level embeddings — ColBERT (Contextualized Late Interaction over BERT) is an embedding practice in which each token within a passage is assigned its own embedding. This granular approach allows individual tokens — especially significant or unique keywords — to exert greater influence on the retrieval process, mirroring the precision of keyword searches while leveraging the contextual understanding of modern BERT models. Despite its higher computational requirements, ColBERT can offer superior retrieval performance by preserving the significance of key terms within the embeddings.

Multi-head RAG approach — Building on the capabilities of transformer architectures, Multi-Head RAG utilizes the multiple attention heads of a transformer to generate several embeddings for each query or passage. Each head can emphasize different features or aspects of the text, resulting in a diverse set of embeddings that capture various dimensions of the information. This method enhances the system's ability to handle complex queries by providing a richer set of semantic cues from which the model can draw when generating responses.

Build structure into your AI stack

Vector search and AI systems are ideal for unstructured knowledge and data, but most use cases could benefit from some structure in the AI stack.

One very clear example of this: if your use case and your users rely on keyword search and exact text matching, then it is probably a good idea to integrate a document store with text search capabilities. It is generally cheaper, more robust,

and easier to integrate classical text search than it is to try to get a vector store to be a highly reliable text search tool.

Knowledge graphs can be another good way to incorporate structure into your AI stack. If you already have, or can build, a high quality graph that fits your use case, then building out some graph functionality, such as graph RAG, can boost the overall utility of your AI system.

In many cases, **our original data set may have inherent structure** that we are not taking advantage of with vector search. It is typical for almost all document structure to be stripped away during the data prep process, before loading into a vector store. HTML, PDFs, Markdown, and most other document types contain structural elements that can be exploited to make our AI systems better and more reliable. In the next section, let's have a look at how this might work.

Add a layer of structure with document linking and tagging

Returning to our case study above, we can exploit the structure of our HTML documents to make our vector search and RAG system better and more reliable. In particular, we can use the hyperlinks in the HTML documents to connect related entities and concepts to ensure that we are getting the big picture via all of the relevant documents in our vector store. See this previous article for an introduction to document linking in graph RAG.

Notably, in our document set, all product names are linked to product pages. Each time one of the three products is mentioned on a page, the product name text is hyperlinked to the corresponding product page. And all of the product pages link to each other.

We can take advantage of this link structure using vector graph traversal and the `GraphVectorStore` implementation in LangChain.

This implementation allows us to easily build a knowledge graph based on hyperlinks between documents, and then traverse this graph to pull in documents that are directly linked to given documents. In practice (and under the hood), we first perform a standard document retrieval via vector search, and then we traverse the links in the retrieved documents in order to pull in more connected documents, regardless of whether they appear “relevant” to the vector search. With this implementation, retrieval fetches both the set of documents that are most

semantically relevant to the query, as well as documents that are directly linked, which could provide valuable supporting information to answer the query.

Re-configuring the retrieval from our use case to traverse the graph of links by one step from each document (`depth=1`), we get the following results from our original query:

Question:

What are some weaknesses of Phrase Flow?

Retrieved documents:

```
['https://phrase.ai/products/phraseflow',  
'https://phrase.ai/ongoingwork',  
'https://phrase.ai/usecases',  
'https://phrase.ai/purpose',  
'https://phrase.ai/products/phrasefactory',  
'https://phrase.ai/products/phraseforge']
```

LLM response:

Phrase Flow is not ideal for documents that need to be factual or for creating a complete Table of Contents. Additionally, it might not be the best choice for tasks that require a lot of thought or structure, as it is more focused on making the language engaging and fluid rather than detailed and organized.

We can see in this output that, even though we still have the initial retrieval set to `k=4` documents returned from vector search, two additional documents were retrieved because they are directly linked from the original document set. These two documents contain precisely that critical information that was missing from the original query results, when we were using only vector search and no graph. With these two documents included, the two warnings about Phrase Flow are available in the retrieved document set, and the LLM can provide a properly informed response.

Within this RAG system with vector graph, the vectors may be lossy, but hyperlinks and the resulting graph edges are not. They provide solid and meaningful connections between documents that can be used to enrich the retrieved document set in a reliable and deterministic way, which can be an antidote to the lossy and unstructured nature of AI systems. And, as AI continues to revolutionize the way we work with unstructured data, our software and data stacks continue to benefit from

exploiting structure wherever we find it, especially when it is built to fit the use case in front of us.

Conclusion

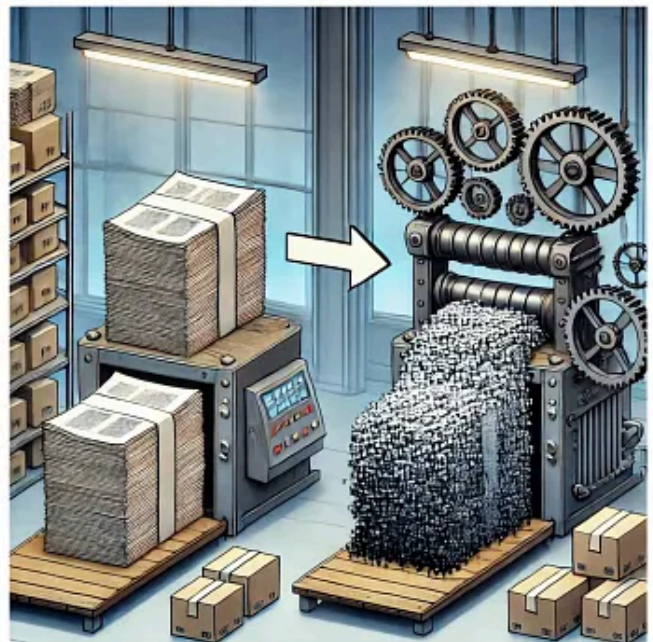
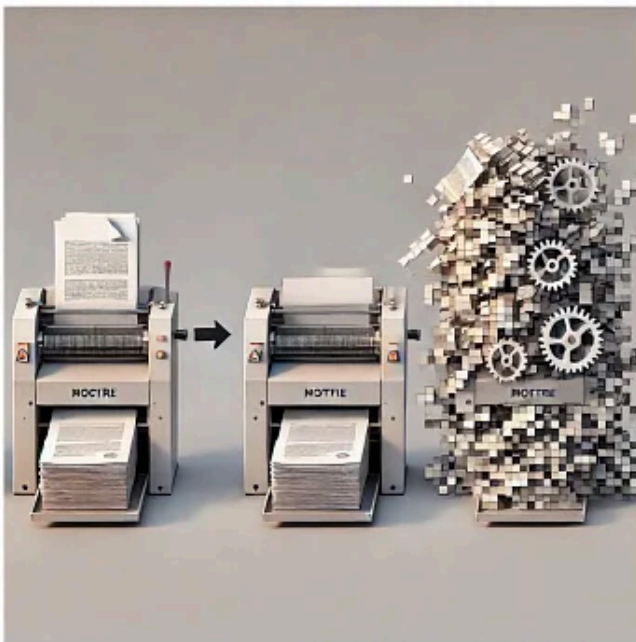
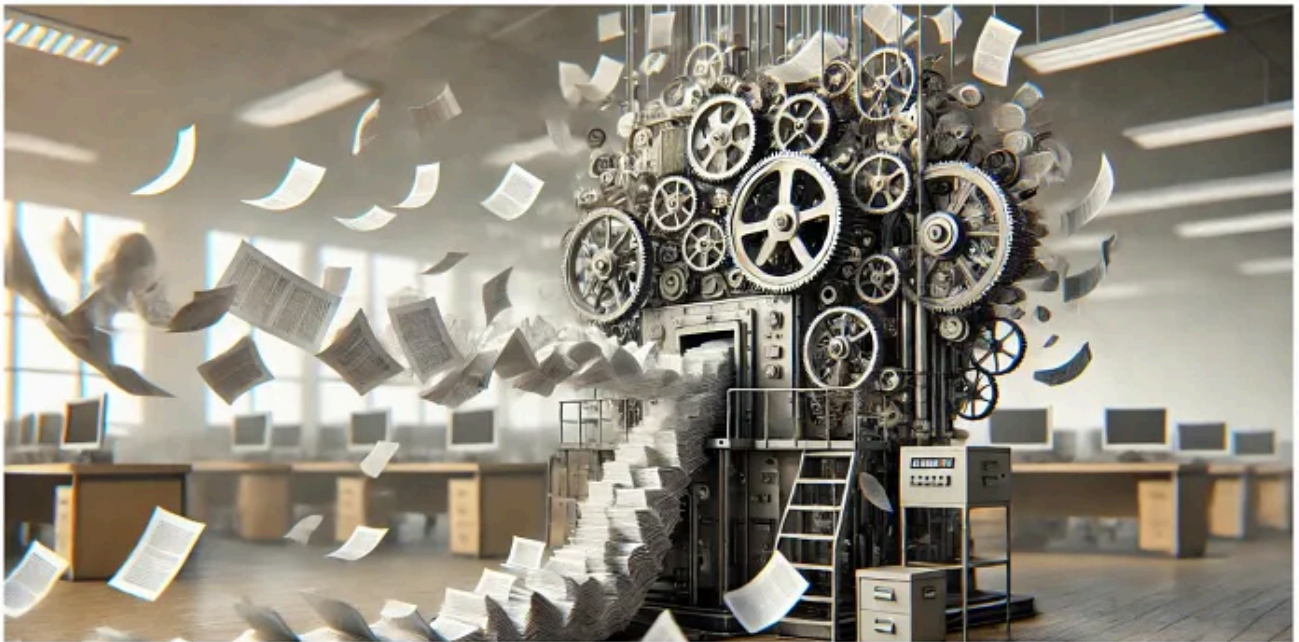
We know that vector embeddings are lossy, in a variety of ways. Choosing an embedding scheme that aligns with your dataset and your use case can improve results and reduce the negative effects of lossy embeddings, but there are other helpful options as well.

A vector graph can take direct advantage of structure inherent in the document dataset. In a sense, it's like letting the data build an inherent knowledge graph that connects related chunks of text with each other — for example, by using hyperlinks and other references that are present in the documents to discover other documents that are related and potentially relevant.

You can try linking and vector graph yourself using the code in [this Colab notebook](#) referenced in this article. Or to learn about and try document linking, [see my previous article](#) or the deeper technical details of [Scaling Knowledge Graphs by Eliminating Edges](#).

by Brian Godsey, Ph.D. ([LinkedIn](#)) — mathematician, data scientist and engineer // works on AI products at [DataStax](#) // Wrote the book [Think Like a Data Scientist](#)

P.S. Just for fun, here are some other attempts at generating cover images for this article. 😎



Generated by the author using DALL-E.



Generated by the author using DALL-E.

Retrieval Augmented

Genai

Knowledge Graph

Graphrag

Editors Pick



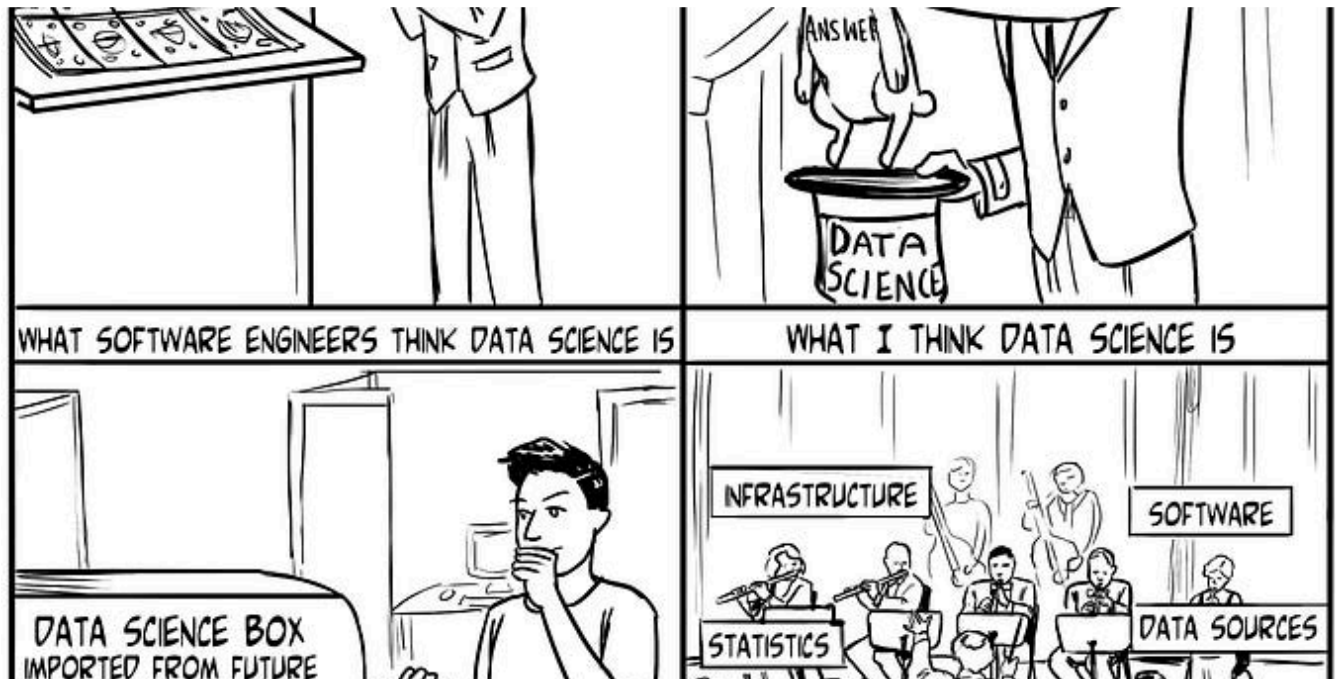
Follow

Written by Brian Godsey

437 Followers · Writer for Towards Data Science

AI @DataStax // mathematics, AI, ML, data sci/eng, writer, runner // wrote Think Like a Data Scientist // briangodsey.com // tw: @brian_godsey

More from Brian Godsey and Towards Data Science



 Brian Godsey in Towards Data Science

Is Data Science Really a “Science”?

Why data science might be a rigorous field different and distinct from its predecessors

Nov 8, 2018 🖱 370





Shaw Talebi in Towards Data Science

Python QuickStart for People Learning AI

A beginner-friendly guide



Sep 8

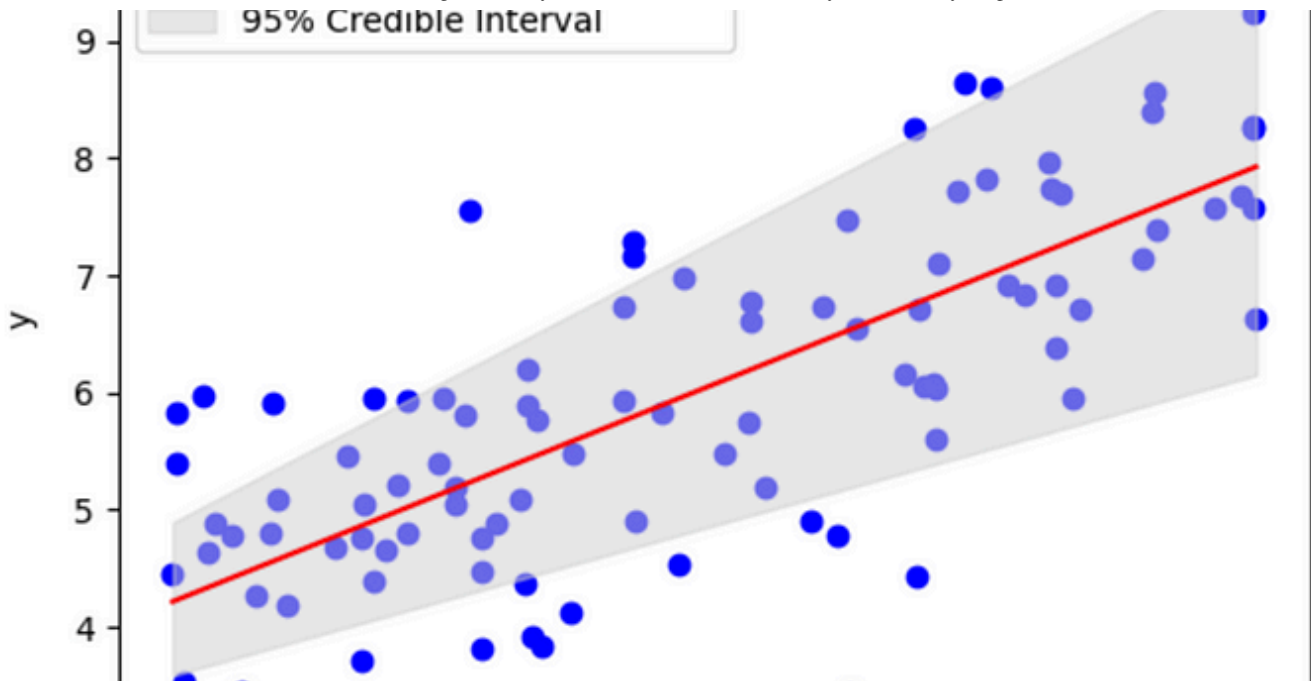


419



4





Samvardhan Vishnoi in Towards Data Science

Bayesian Linear Regression: A Complete Beginner's guide

A workflow and code walkthrough for building a Bayesian regression model in STAN

Sep 14

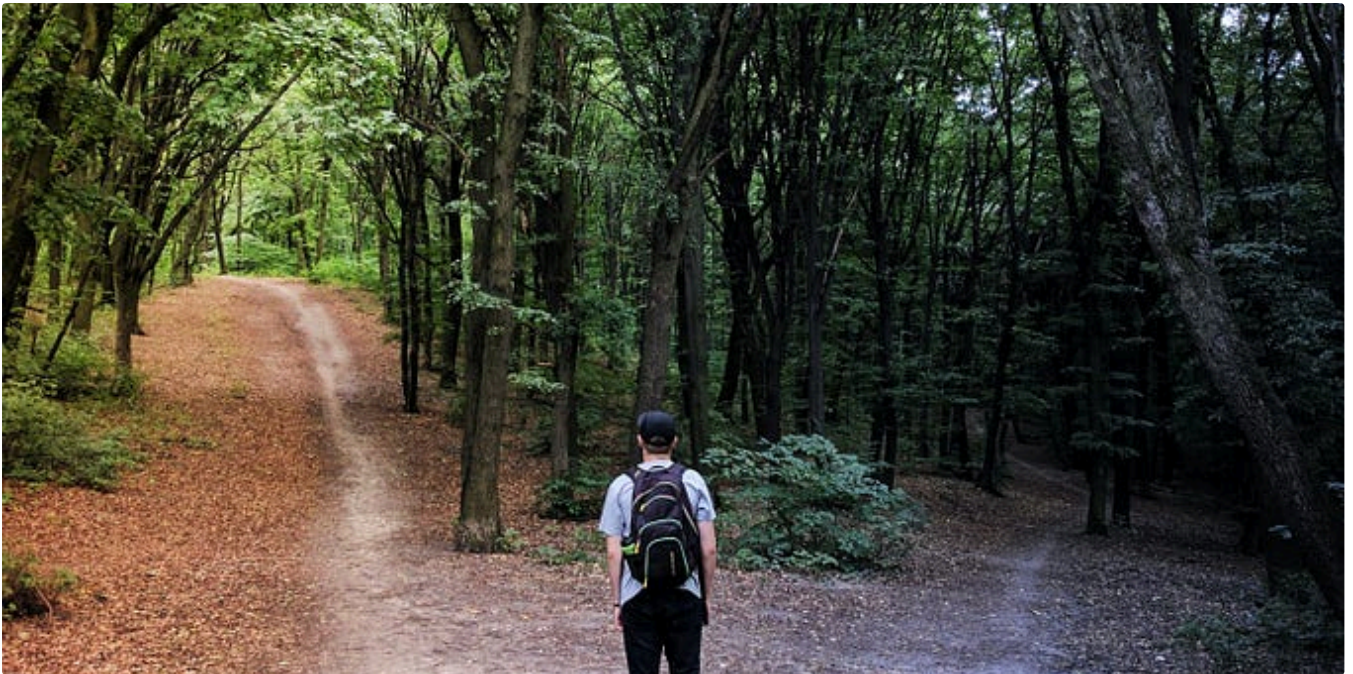


257



3





 Brian Godsey  in Towards Data Science

The Bayesians Have Won Data Science

Frequentists want to repeat the experiment

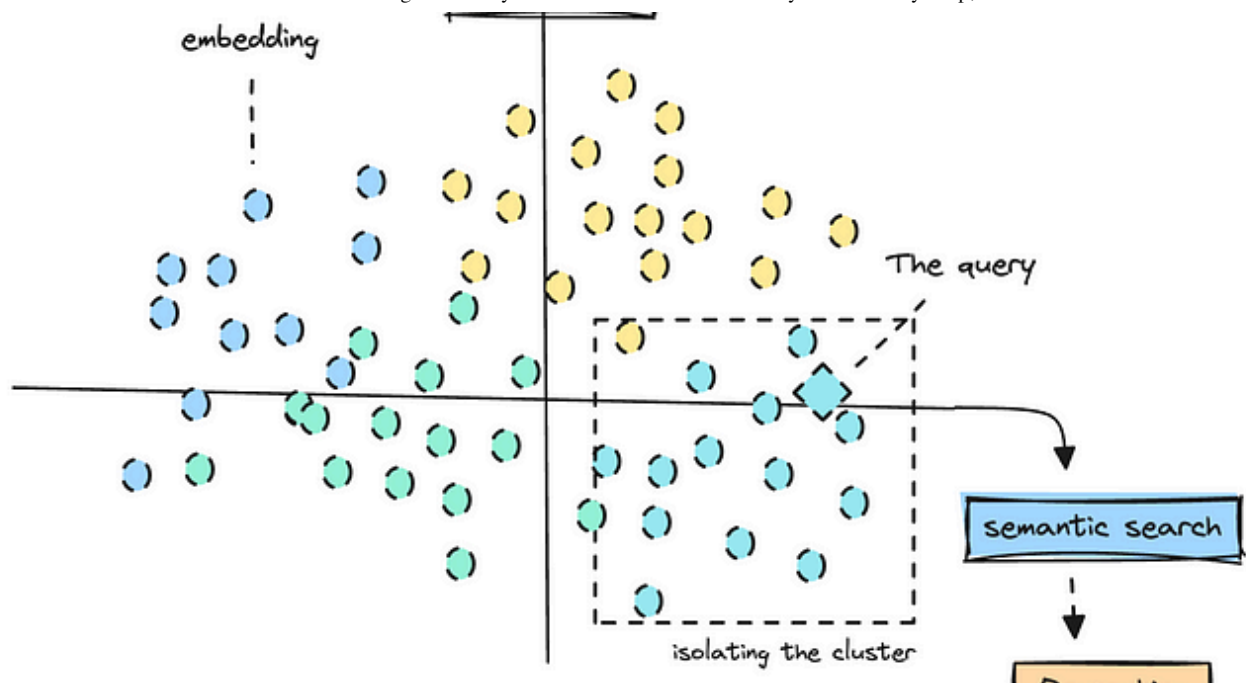
Oct 25, 2021  454  7



See all from Brian Godsey

See all from Towards Data Science

Recommended from Medium



 Ida Silfverskiöld in Towards Data Science

Working with Embeddings: Closed versus Open Source

Using techniques to improve semantic search

★ 5d ago 🖱 606 💬 9



 Dr. Ashish Baman  in Level Up Coding

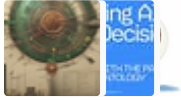
'MathPrompt' Embarassingly Jailbreaks All LLMs Available On The Market Today

A deep dive into how 'MathPrompt' works, why it is so effective, and why it needs early patching to prevent harmful LLM content generation

★ 5d ago 🖱️ 616 💬 8



Lists



data science and AI

40 stories · 251 saves



The New Chatbots: ChatGPT, Bard, and Beyond

12 stories · 468 saves



Natural Language Processing

1733 stories · 1305 saves



Business

41 stories · 144 saves



 Tomaz Bratanic in Neo4j Developer Blog

Limitations of Text Embeddings in RAG Applications

Learn how to overcome them using knowledge graphs and structured tools

5d ago 🖱️ 179 💬 2



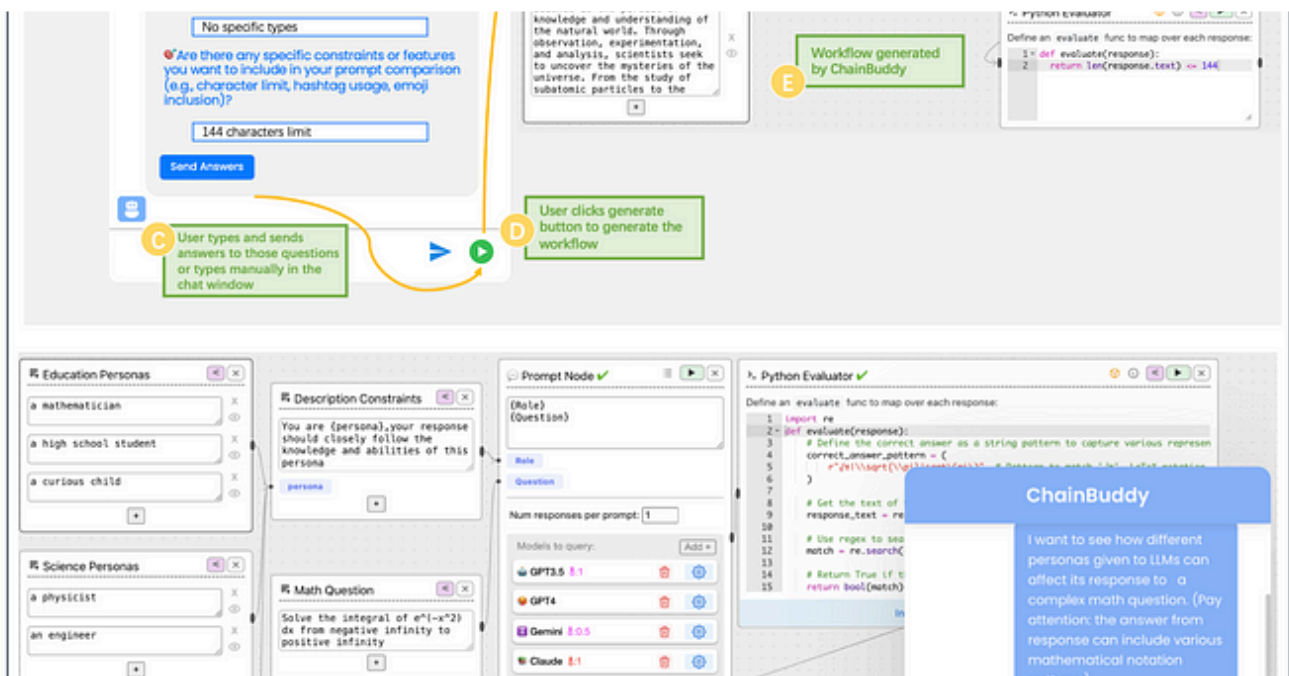


 Amos Gyamfi

The 6 Best LLM Tools To Run Models Locally

Running large language models (LLMs) like ChatGPT and Claude usually involves sending data to servers managed by OpenAI and other AI model...

Aug 28  128  2



 Cobus Greyling

ChainBuddy

An AI Agent System for Generating LLM Pipelines.

6d ago

106

Image	College-level Problems and Mathematical Reasoning MMM-U (text, CoChat Co L, monolingual accuracy)	50.7	60.3	50.2	59.4
	MMM-U-Pro, Standard (text, CoChat Co L, monolingual accuracy)	33.0	45.2	27.3	42.3
	MMM-U-Pro, Vision (text, CoChat Co L, monolingual accuracy)	23.7	33.8	20.1	36.5
	MathVista (text, CoChat Co L, monolingual accuracy)	51.5	57.3	46.4	56.7
	ChartQA (text, CoChat Co L, monolingual accuracy)	83.4	85.5	81.7	—
	Ai2 Diagram (text, CoChat Co L, monolingual accuracy)	91.1	92.3	86.7	—
	DocVQA (text, CoChat Co L, monolingual accuracy)	88.4	90.1	88.8	—
	Visual Question Answering VQA-v2 (text, CoChat Co L, monolingual accuracy)	75.2	78.1	—	—
	General MMLU (text, CoChat Co L, monolingual accuracy)	73.0	86.0	75.2 (5-shot)	82.0
Text	Math MATH (text, CoChat Co L, monolingual accuracy)	51.9	60.0	38.9	70.2
	WordNet (text, CoChat Co L, monolingual accuracy)	—	—	—	—

Thomas Reid

in AI Advances

Testing the new Llama 3.2 90B Vision model from Meta

How does it stack up to Pixtral and Qwen 2 VL

4d ago

223

See more recommendations